

testing in the model learning stage needs to ensure that model implementation using either a standard machine learning library/framework or self-written code does not contain any implementation bugs. Testing also needs to ensure that the values of the hyper-parameters are appropriately chosen.

The third stage in the ML life cycle is model verification. This is when the model needs to provide evidence that it is meeting the requirements it was designed to satisfy. In many cases, this could be as simple as verifying that the model meets the performance criteria it was designed to fulfill, namely its precision being greater than X per cent and recall being greater than Y per cent on a held out test set. This is typically the case for many of the current crop of ML applications, such as recognising licence plates, automatic machine translation or predicting stock market prices. However, in cases where the system's impact on human life is much greater, more stringent verification criteria may be in place.

Consider an ML application that is replacing human doctors in the radiology department, analysing X-ray images and detecting tumors. Alternatively, consider an application, which is used to help judges decide whether people can be paroled or whether they are likely to commit a second crime if they are released on parole. Such decisions taken by ML based applications have an enormous impact on human lives, and hence any possibility of the system making erroneous decisions needs to be analysed and addressed before it is deployed in the real world. Therefore, model verification is critical for such applications.

Model verification can be done either through verification testing or through formal mathematical verification. Testing based verification involves testing not just the expected operational region inputs for the ML application, but also the adversarial region and the failure region inputs. Adversarial region inputs are those that are correctly predicted by the ML application but return a wrong result when perturbed imperceptibly in a manner indistinguishable by a human from the original unperturbed input. Adversarial training has been advocated as a means of avoiding adversarial failures during the second stage of model training. During model verification, further adversarial testing can be done to ensure that the trained model is robust to adversarial inputs.

Failure testing ensures that the system handles inputs in regions where it is expected to fail, efficiently, without issues. For instance, if an X-ray of extremely poor quality is presented to the AI radiological application, it should reject the input instead of classifying it, while providing an explanation for doing so.

Formal verification of the ML model is generally quite complex and requires considerable effort, given the high dimensional input space and inner states of the ML model. While there is some work going on in applying software verification approaches such as abstract interpretation and Satisfiability Modulo Theory (SMT) to ML applications, these efforts are still in the nascent stages.

Readers who would like to know more about formal approaches to neural network verification can refer to the paper by Huang et al. available at <https://arxiv.org/pdf/1610.06940.pdf>.

Once an ML application crosses the first three stages meeting the necessary verification criteria, and its performance is satisfactory as per the requirements, the final stage is the deployment stage of the ML model. At this point it gets integrated with other required software components to form a working operational system. The deployment stage requires monitoring of the deployed model to ensure that it continues to operate satisfactorily and to identify any operational failures well ahead of time. It also requires periodic maintenance and upgradation of the model since inputs from real work are dynamic and change over time; the model needs to adapt to these changes. This can be done through offline maintenance, upgradation or continuous active learning.

One of the first and most important things is to ensure that the model's performance, which has been tested during the earlier stages, continues to remain acceptable during the deployment stage as well, when it is used in the real world. We also need to ensure that the input data coming to the model continues to be of the same distribution as the training data set. If the deployment stage input data varies considerably from the training data set distribution, it is called data distributional drift. This should be detected early, and the model must retrain if adequate performance is to be maintained.

Given that model performance is measured on aggregate performance scores (instead of determining correctness on each input, models are scored based on their aggregate performance, namely, the overall F1 score on the test data set), minor performance glitches on specific subsets of inputs can go without getting detected. Think of a self-driving car system during deployment, for which the computer vision module performs poorly when detecting grey coloured oncoming trucks on white snow-covered concrete roads in the poor light of winter evenings. While this specific input region would be a very small subset of all possible inputs, poor performance on this specific input region during deployment can lead to unsafe overall system behaviour and to the possible loss of human lives. So, deployment testing needs to cover not only aggregated performance measures, but also test specific input regions of interest to ensure that the model has not degraded in performance on specific subsets of the input space. We will discuss more on this topic in next month's column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Wishing all our readers happy coding until next month! **END** 🐧

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing.